

12. Transformace a zobrazení 3D polygonálních modelů, principy programovatelného vykreslovacího řetězce

12. 3D transformace a vykreslovací řetězec

SZZ okruh 12 (FIT BUT). Transformace = změna pozice vrcholů nebo souřadnicového systému.

Shrnutí

Transformace

- **Lineární** (zachovává lineární kombinaci): měřítko, rotace, zkosení.
- **Afinní** (zachovává kolinearitu a dělicí poměr): + posunutí; lineární následovaná posunem.
- **Homogenní souřadnice** — přidaná váha **w** (u afinních $w = 1$) → jednotný **maticový zápis** všech transformací, snadné **skládání** a perspektivní projekce.
- Skládání závisí na pořadí; první transformace musí být **nejblíže** transformovanému bodu.

Projekce

- **Paralelní (ortografická)** — zachovává rovnoběžnost, vzdálenost neovlivní velikost; CAD.
- **Perspektivní (středová)** — paprsky se sbíhají do středu projekce, vzdálenost zmenšuje objekt; hry, VR. Kamera se obvykle zafixuje do počátku a hýbe se scénou.

Realistické zobrazení

- **Ray tracing** — zpětné sledování paprsku od kamery; primární, stínové, sekundární paprsky; výpočetně náročné, závisí na poloze kamery.
- **Radiozita** — globální osvětlení, šíření energie mezi ploškami (BRDF, konfigurační faktory); **nezávisí** na poloze kamery; lze kombinovat s ray tracingem.

Programovatelná zobrazovací pipeline

1. **Vertex Assembly** — sestavení atributů vrcholů (stride, offset).
2. **Vertex Shader** (programovatelný) — $\text{model} \times \text{view} \times \text{projekční matice}$ (model space → clip space).
3. **Primitive Assembly** — sestavení trojúhelníků ze 3 vrcholů.
4. **Clipping/Culling** — ořez na view frustum, zahození neviditelných.
5. **Perspective Division** — z homogenních na kartézské podle hloubky.
6. **Viewport Transformation** — NDC $[-1, +1]$ → rozlišení okna.
7. **Rasterization** — produkuje **fragmenty**.
8. **Fragment Shader** (programovatelný) — obarvení/textura, interpolace přes **barycentrické souřadnice**.
9. **Per-Fragment Operations** — **hloubkový test** (z-buffer) a **blending** (např. alpha).

[[media/szz-12/media/image4.png]] Programovatelný vykreslovací řetězec (rendering pipeline).

Souvislosti

Rasterizace trojúhelníků navazuje na [[okruhy/11-2d-vektorova-grafika | 2D rasterizaci]]; prvky GUI staví na této pipeline viz [[okruhy/13-graficka-uzivatelska-rozhrani]]; matice a vektory řeší lineární algebra.

Časté otázky u zkoušky

Na co se u tohoto okruhu typicky ptají. Plné odpovědi níže.

Vykreslovací řetězec □ [[#Programovatelná zobrazovací pipeline]] - Části pipeline? □ vertex assembly □ vertex shader □ primitive assembly □ clipping □ perspective division □ viewport □ rasterizace □ fragment shader □ per-fragment (depth test, blending).

Homogenní souřadnice □ [[#Transformace]] - Co a proč? □ Kartézské souřadnice + váha w ; umožní vyjádřit i posun maticí a skládat transformace násobením; u bodů $w = 1$. - Skládání transformací? □ Násobení matic; záleží na pořadí (nekomutativní), první transformace nejbliž bodu (např. rotace kolem bodu = posun □ rotace □ posun zpět). - Lineární vs. afinní? □ Lineární zachovává sčítání a násobení skalárem (měřítko, rotace, zkosení); posun není lineární □ afinní (proto homogenní souřadnice).

Projekce a reprezentace □ [[#Projekce]] - Paralelní vs. perspektivní? □ Paralelní zachovává rovnoběžnost, velikost nezávisí na vzdálenosti (CAD); perspektivní sbíhá do středu, vzdálené se zmenšuje (hry, VR). - Proč *polygony* (trojúhelníky)? □ Vždy konvexní, efektivní v HW; alternativy CSG, voxely.

Plné znění (ke studiu)

Geometrická transformace

změna pozice **vrcholů** objektů v **aktuálním** souřadnicovém systému nebo **změna souřadnicového** systému

Lineární transformace

Lineární transformace **zachovává lineární kombinaci** (vektory je možné aplikovat **zároveň nebo po jednom** a výsledek bude vždy stejný). Pro **libovolné** dva vektory a **skalár** platí:

Mezi lineární transformace patří **měřítko** (zvětšení, zmenšení), **rotace** a **zkosení**. [[media/szz-12/media/image11.png]]

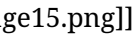
Afinní transformace

Afinní transformace zachovává **kolinearitu a dělicí poměr** (body ležící na přímce budou ležet na přímce - v jednom bodě; i po zobrazení). Všechny základní geometrické operace (**měřítko** **S_c** , **rotace** **R** , **zkosení** **S_h** i **posunutí** **T**) jsou afinní. Lze ji vyjádřit jako lineární transformaci následovanou posunem. Každá lineární transformace je současně i afinní.

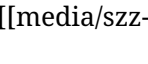
Homogenní souřadnice (váha)

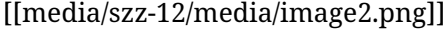
Ke standardním **kartézským** souřadnicím (x, y ve 2D a x, y, z ve 3D) je přidána jedna navíc - souřadnice **w** (váha bodu, u **afinních** je **$w = 1$**). Umožňují pracovat se **všemi** druhy základních **transformací** jednotně pomocí **maticového zápisu**. Maticový zápis umožňuje provádět jednoduché skládání transformací.

Důvody použití:

- Jednotná reprezentace základních transformací pomocí maticového zápisu.
- Umožňují skládání transformací. 
- Realizace perspektivní projekce.

Transformace

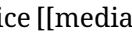
- **Posunutí** (translate): Ve **3D** stačí rozšířit o řádek a sloupec s **dz**, resp. **-dz**. 



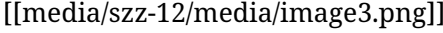
- **Změna měřítka** ve 3D (scale): $P' = P \cdot S$, ve 3D stačí rozšířit o řádek s **Sz**, resp. **1/Sz**.

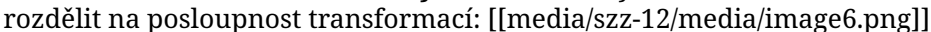
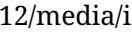


- **Zkosení: 1** matice pro zkosení ve **2D**, **3** matice pro zkosení ve směrech YZ, XZ a XY ve **3D**. 

- **Rotace kolem počátku souřadného systému:** Ve 3D 3 matice 





- **Rotace ve 3D kolem obecné osy:** dána směrovým vektorem **v** a bodem umístění **P**, je třeba rozdělit na posloupnost transformací: 
 - 1. Posunutí osy rotace do počátku souřadného systému
 - 2. Sklopení posunuté osy do jedné ze souřadných rovin
 - 3. otočení sklopené osy do jedné ze souřadných os (např. do X)
 - 4. Provedení požadované rotace o úhel ω kolem příslušné osy (zde X)
 - 5. Vrácení osy rotace do původní polohy 

Zkráceně musíme nejdřív **obecnou osu dostat do pozice jedné z os souřadného systému**, provést otočení a vrátit osu do původního místa.

Skládání transformací

U skládání transformací závisí na jejich pořadí, např. **posunutí, rotace, zvětšení**.

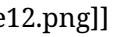
- **zápis bodu v řádku:** $P' = P \cdot T \cdot R \cdot S$, matici M souhrnné transformace získáme jako $M = T \cdot R \cdot S$ a $P' = P \cdot M$.
- **zápis bodu ve sloupci:** $P' = S \cdot R \cdot T \cdot P$, matici M souhrnné transformace získáme jako $M = S \cdot R \cdot T$ a $P' = M \cdot P$.

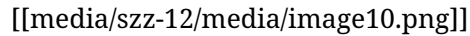
V obou případech musí být první transformace v pořadí **nejblíže transformovanému bodu**.

Zobrazení 3D polygonálních modelů

Zobrazení **3D** objektů provádíme většinou na **2D** obrazovku **projekcí** (transformací z 3D do 2D, to znamená ztrátu dat), obrazovka je tedy **průmětna**. Promítání provádíme pomocí paprsků, tzv. **projekčních paprsků**. V počítačové grafice se většinou rasterizují všechny objekty **pomocí trojúhelníku** (konvexnost, lze dobře akcelarovat v HW).

Prvky 3D scény

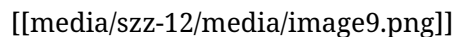
- **near clip plane:** určuje minimální hloubku zobrazovaných objektů,
- **far clip plane:** udává maximální hloubku zobrazovaných objektů, 
- **viewing frustum:** prostor v modelovaném prostředí, který se objeví na obrazovce,
- **viewpoint:** místo, ze kterého scénu pozorujeme (kamera)
- **světlo:** bodové nebo plošné, umístění rozhoduje o viditelnosti objektů (barvy, stíny, ...),
- **modelované objekty.**

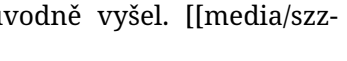


Paralelní projekce (rovnoběžná, ortografická)

- **zachovává rovnoběžnost hran**
- **vzdálenost** průmětny **neovlivňuje velikost** průmětu (při změně vzdálenosti objektu se nemění jeho velikost)
- **kolmé promítání** - paprsky jsou **kolmé na průmětnu**
- použití: technické **CAD** aplikace, výkresová dokumentace

Perspektivní projekce (středová)



- **nezachovává rovnoběžnost hran**
- použití ve **hrách, VR** a jinde
- **vzdálenost průmětny** od objektu **ovlivňuje velikost průmětu** (se vzdáleností objektu se objekt zmenšuje)
- nelineární středová projekce: paprsky **vycházejí z 1 bodu - středu projekce**
- pro jednodušší manipulaci se **kamera** obvykle **zafixuje** do počátku souřadného systému a **hýbe** se (pomocí transformačních matic) se **scénou**.
 - **Geometrický princip projekce:** z vrcholů trojúhelníku se do středu projekce (tj. střed souřadného systému, kam je umístěna i kamera) vrhnou paprsky (u perspektivní projekce se všechny paprsky sbíhají do středu projekce). V tom místě, kde paprsek protnul projekční rovinu, se promítne bod, ze kterého paprsek původně vyšel. 

RayTracing

Metoda pro realistické zobrazování. Funguje na principu zpětného sledování cest paprsku od kamery ke zdroji světla. Hledá se množství světla, které paprsek přináší.

Paprsky dělíme na:

- **Primární:** Vychází z kamery, mají společný počátek nebo jsou rovnoběžné
- **Stínové:** Z místa dopadu paprsku do každého světla, zajímá nás, jestli jej **vidíme nebo nevidíme**, podle toho následně spočítáme **stín**.
- **Sekundární:** Vznikají **odrazem a lomem** z míst **dopadů primárních** a sekundárních paprsků. Jsou chaotičtější než primární paprsky.

Výpočetně je RayTracing velmi náročný, lze optimalizovat například ohledem na to, že **sekundární** paprsky mají **menší vliv** na výsledný obraz, **intenzita světla** se snižuje se **čtvercem** vzdálenosti, **snížit rozlišení**. Výpočet scény je **závislý na poloze kamery**.

Radiozita

Metoda **globálního osvětlení scény**. Řeší **šíření energie**, objekty mohou být sekundárními zdroji světla (odraz).

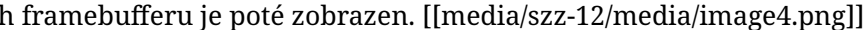
- Vychází ze zákona zachování energie, vyžaduje energeticky uzavřenou scénu.
- Scéna musí být reprezentovaná **polygonálním modelem** (jiné modely - CSG, B-rep, Drátové modely).
- Vychází z dvousměrové distribuční **funkce BRDF**.
- Před vlastním výpočtem je třeba **polygony** ve scéně **rozdělit na malé plošky** a spočítat **konfigurační faktory** (**vliv** každé **plošky** na každou **jinou plošku** ve scéně).
- Nedokáže pracovat s průhlednými objekty.
- Výpočet scény **není závislý na poloze kamery**.
- V praxi lze **kombinovat s RayTracing**.

Principy programovatelného vykreslovacího řetězce (zobrazovací pipeline)

zobrazovací pipeline je tvořena těmito částmi:

1. **Vertex Assembly (Vertex Puller)**: je zařízení na grafické kartě, které se stará o **sestavení vrcholů**. Vertex puller je tvořen **čtecími hlavami**, každá konstruuje jeden atribut vrcholu (pozice, normála, souřadnice textury, barva, ...). Čtecí hlavy se pohybují s krokem (**stride**) a mají nějaký posun (**offset**). Data čtou z pole bytů.
2. **Vertex Shader** (programovatelný): Provádí zpracování vrcholů z Vertex Assembly. Jedná se o násobení **modelovou maticí**, **view maticí** a **projekční maticí**. Vstupem jsou vrcholy v **model space**, výstupem vrcholy v **clip space**.
3. **Primitive Assembly**: je jednotka, která sestavuje trojúhelníky. **Čeká na 3** po sobě jdoucí **vrcholy** z vertex shaderu a **sestaví trojúhelník**. Lze na to také nahlížet tak, že Primitive Assembly dostane příkaz vykreslit třeba 4 trojúhelníky. Jednotka tak spustí Vertex Shader 12x, který takto spustí 12x Vertex Assembly.
4. **Clipping/Culling**: provádí **ořez prostoru** na view frustum. Trojúhelníky na **hranicích** musí ořezat (může vést na rozdělení na 2) a zahazuje trojúhelníky, které nejsou vůbec vidět (odvrácené, překryté).
5. **Perspective Division**: provádí převod z homogenních souřadnic na kartézských na základě hloubky trojúhelníků (dělením - co je daleko se zmenší více, co je blízko se zmenší méně).
6. **Viewport Transformation**: Převádí souřadnice z NDC (normalized device coordinates, -1, +1) na rozlišení okna, aby se mohla provést rasterizace.
7. **Rasterization**: rasterizuje připravené trojúhelníky a produkuje **fragmenty** (čtvercové úlomky trojúhelníku - **pixels**, které se nakonec **zapiší do framebufferu**).
8. **Fragment Shader** (programovatelný): **obarvuje fragmenty** (aplikuje textury na fragmenty) uvnitř trojúhelníka (střed pixelu musí ležet uvnitř). Pro konkrétní fragment (pixel) trojúhelníku se spočítají **barycentrické souřadnice** a použijí se pro **interpolaci** všech atributů z **vrcholů**.
9. **Per-Fragment Operations**: jedná se o dvě operace: **hloubkový test** a **blending** (a další testy a operace).

- **Hloubkový test** (depth test) se stará o **zahazování fragmentů**, které jsou **hlouběji** než to, co už se **vyrasterizovalo**, **naopak** pokud je **hloubka** nového fragment **menší**, je jeho **barva a hloubka zapsána do framebufferu**.
- **Blending** místo přepsání barvy ve framebufferu je **míchá**. Existuje mnoho způsobů realizace, např. pomocí **průhlednosti (alpha blending)**.

Obsah framebufferu je poté zobrazen. 

 style="width:6.25in;height:2.17708in" />

Jiná reprezentace objektů

- **CSG** (Constructive solid geometry) - skládání jednoduchých 3D primitiv (koule, krychle, kvádry, válce, ...).
- Pomocí objemových jednotek - **voxely** (analogie pixelu ve 3D).
- **Implicitní plochy** - potenciální pole kolem nějaké kostry.

Zdroje

- SZZ okruh 12 — studijní materiály FIT BUT (szz-12.docx). Další obrázky: media/szz-12/.

Navigace: [\[\[index| • Přehled okruhů\]\]](#) · [\[\[okruhy/11-2d-vektorova-grafika|11. 2D vektorová grafika\]\]](#) · další: [\[\[okruhy/13-graficka-uzivatelska-rozhrani|13. Grafická uživatelská rozhraní\]\]](#) [□](#)